A

Project Report on

Architecture Rules of Thumb for DevOps

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

P. NAVYA

24K81A05Q1

Under the Guidance of

Mr.M.Dileep Kumar

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+,ISO 9001-2008 Certified

Dhulapally,Secunderabad-500 100

www.smec.ac.in

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+,ISO 9001-2008 Certified

Dhulapally,Secunderabad-500 100

www.smec.ac.in

Certificate

This is to certify that the project entitled "**Architecture Rules of Thumb for DevOps**" is being submitted by **P. Navya (24K81A05Q1)** in fulfilment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** is recorded of Bonafide work carried out by them. The result embodied have been verified and found satisfactory.

Project Internal Examiner

Mr.M.Dileep Kumar
Assistant Professor
Department of CSE

Signature of HOD

Mr. D. KRISHNA KISHORE
Associate Professor and HOD
Department of CSE

Internal Examiner

External Examiner

St. MARTIN'S ENGINEERING COLLEGE

SHAPING VIKSIT BHARAT'S BRIGHTEST MINDS

UGC Autonomous

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+,ISO 9001-2008 Certified

Dhulapally,Secunderabad-500 100

www.smec.ac.in

COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, the student of "**Bachelor of Technology in Department of Computer Science and Engineering**", session: 2024-2028, **St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad** , here by declare that the work presented in this project work is **Architecture Rules of Thumb for DevOps** is the outcome of our own Bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

P. NAVYA – 24K81A05Q1

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, I would like to express my deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express my deep sense of gratitude and indebtedness to **Dr. K. RAVINDRA,** Director, Principal, Professor St. Martin's Engineering College Dhulapally, for permitting us to undertake this project.

I am also thankful **Mr.D. KRISHNA KISHORE,** Head of the Department of Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout my project.

I would like to express my sincere gratitude,thanks and indebtedness to my project supervisor **Mr.M.Dileep Kumar** Assistant Professor, Department of Computer Science and Engineering, St. Martins Engineering College, Dhulapally, for his support and guidance throughout project.

Finally, I express thanks to all those who have helped me in successfully completing this project. Furthermore, I would like to thank my family and friends for their moral support and encouragement. I express thanks to all those who have helped us in successfully completing the project.

UGC AUTONOMOUS

P. NAVYA – 24K81A05Q1

CONTENTS

1. ABSTRACT	**1**
2. INTRODUCTION	**2**
3. LITERATURE SURVEY	**3**
4. SYSTEM REQUIREMENT	**5**
5. FLOW CHART	**6**
6. ALGORITHM	**7**
7. SOURCE CODE	**8**
8. EXPERIMENTAL RESULTS	**10**
9. CONCLUSION	**11**
10.FUTURE ENHANCEMENT	**12**
11.REFERENCES	**13**

1. ABSTRACT

The project “Architecture Rules of Thumb in DevOps” focuses on applying fundamental architectural principles and best practices to improve the software development and deployment process. DevOps combines software development and IT operations to deliver software efficiently, reliably, and continuously. Architecture rules of thumb provide simple guidelines for designing systems that are scalable, secure, maintainable, and easy to manage. This project demonstrates the use of these principles through a sample Python application integrated with GitHub and GitHub Actions. The proposed system follows a Continuous Integration and Continuous Delivery (CI/CD) workflow in which developers upload source code to a GitHub repository, and automated workflows execute tests to verify the correctness of the application. The project emphasizes important practices such as automation, modular design, early testing, secure configuration, error handling, consistent environments, and continuous monitoring. These practices help reduce manual effort, detect software defects at an early stage, and improve the overall quality of the application. Furthermore, the project explains how tools such as Git, GitHub Actions, Docker, and monitoring systems can support the implementation of DevOps architecture principles. The system can be extended with automated deployment, cloud integration, dependency security scanning, and application performance monitoring. The main objective of this project is to demonstrate how architecture rules of thumb can guide the development of reliable and maintainable software systems while improving collaboration between development and operations teams.

UGC AUTONOMOUS

2. INTRODUCTION

DevOps is a combination of Development and Operations that helps software development teams build, test, deploy, and maintain applications efficiently. It focuses on improving collaboration between developers and operations teams through automation, continuous integration, continuous delivery, and continuous monitoring. DevOps practices help organizations deliver software faster, reduce errors, and improve the reliability of applications.

Architecture Rules of Thumb in DevOps refers to a set of simple guidelines and best practices that help developers design and manage software systems effectively. These guidelines support important qualities such as scalability, reliability, security, maintainability, and performance. Following these principles helps teams make better architectural decisions and avoid common problems during software development and deployment.

This project demonstrates the application of architecture rules of thumb using a sample Python application, GitHub, and GitHub Actions. The source code is stored in a GitHub repository, and automated workflows are used to test the application whenever code changes are uploaded. This process helps identify errors early and reduces the need for manual testing. Additional practices, such as modular design, secure configuration, consistent environments, and monitoring, are also considered to improve the overall system.

The main purpose of this project is to understand how architectural principles can be integrated into a DevOps workflow to create reliable, secure, and maintainable applications. By implementing these guidelines, the project demonstrates how automation and good architectural decisions can improve software quality, simplify maintenance, and make the software delivery process more efficient.

UGC AUTONOMOUS

3. LITERATURE SURVEY

The concept of Architecture Rules of Thumb in DevOps is based on established software engineering principles and modern practices for developing, testing, deploying, and maintaining software applications. Various tools, methodologies, and architectural approaches have been introduced to improve software quality, system reliability, automation, and collaboration between development and operations teams.

Traditional software development methods often depend on manual testing, separate development and operations teams, and lengthy deployment processes. These approaches can lead to delays, configuration errors, and difficulties in maintaining applications. DevOps addresses these challenges by integrating development and operations activities and encouraging automation throughout the software delivery lifecycle.

Continuous Integration and Continuous Delivery (CI/CD) are important practices in modern DevOps environments. Continuous Integration allows developers to merge code changes frequently and run automated tests to identify errors early. Continuous Delivery prepares software for release through an automated pipeline. Tools such as GitHub Actions help implement these practices by automatically building and testing applications whenever code changes are submitted.

Modular architecture is another important principle discussed in software engineering literature. Dividing an application into smaller, independent components makes the system easier to develop, test, maintain, and update. Similarly, containerization tools such as Docker help maintain consistent environments across development, testing, and deployment stages, reducing problems caused by differences in software configurations.

Security and monitoring are also essential parts of DevOps architecture. Secure configuration, dependency checking, and appropriate access controls help protect applications and sensitive information. Monitoring tools help identify errors, track application performance, and support timely troubleshooting. These practices improve the reliability and availability of software systems.

From the literature, it can be understood that applying architecture rules of thumb together with DevOps practices helps organizations improve automation, maintainability, security, and software delivery speed. This project applies these principles to a sample Python application using GitHub and GitHub Actions, demonstrating how automated testing and structured development workflows can support the creation of reliable and maintainable software.

4. SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or equivalent and above.
- RAM: Minimum 4 GB; 8 GB recommended.
- Hard Disk: At least 10 GB of available storage.
- System: Desktop computer or laptop.
- Internet Connection: Required for GitHub access and cloud-based workflow execution.

Software Requirements

- Operating System: Windows 10/11, Linux, or macOS.
- Programming Language: Python 3.12 or compatible version.
- Code Editor: Visual Studio Code.
- Version Control System: Git.
- Code Repository: GitHub.
- CI/CD Tool: GitHub Actions.
- Testing Framework: Pytest.
- Optional Tool: Docker for containerization.

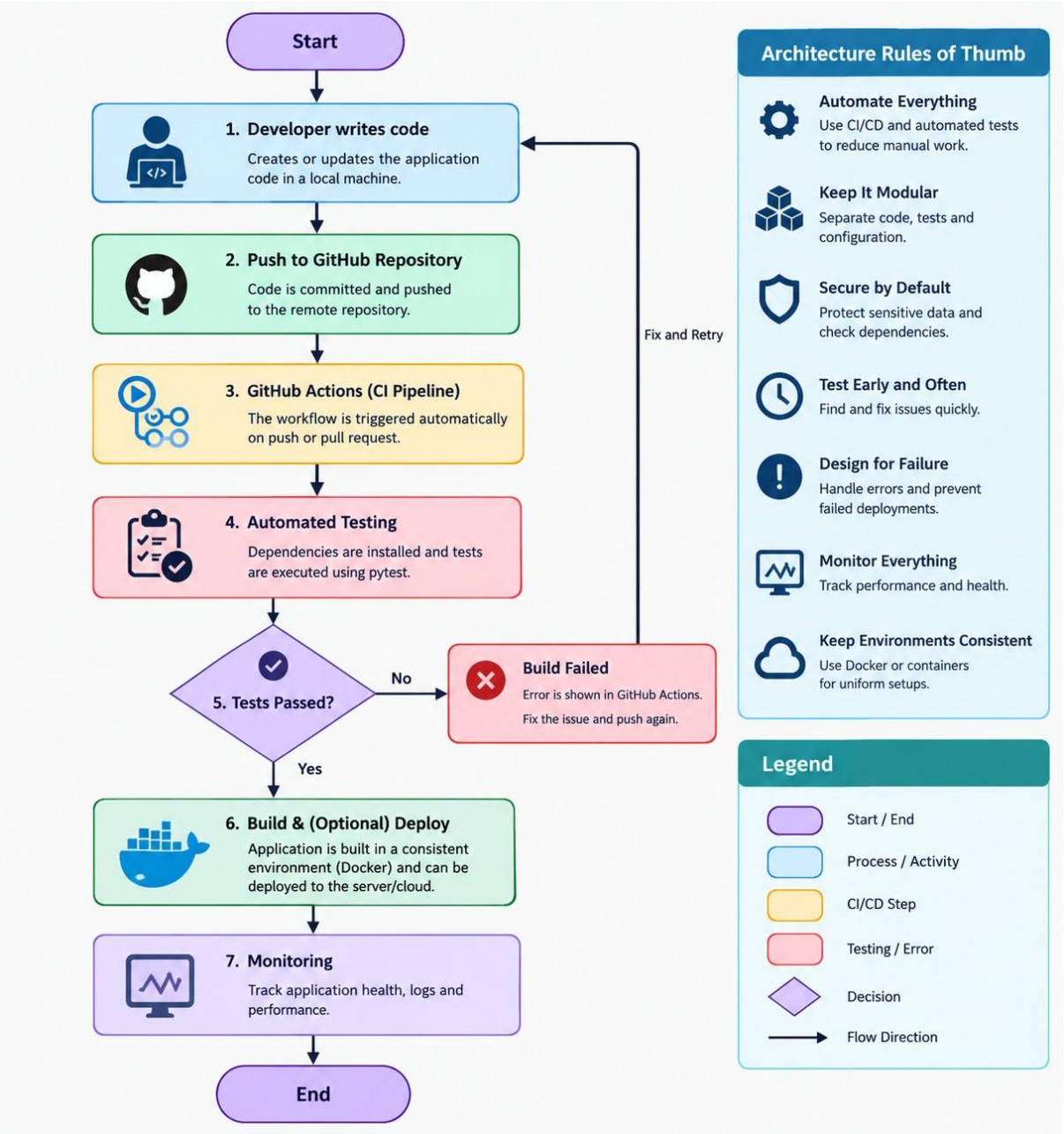
Functional Requirements

- The system should allow developers to create and modify application code.
- The source code should be stored and managed in a GitHub repository.
- The CI workflow should automatically run when code is pushed or a pull request is created.
- The system should execute automated tests to verify application functionality.

Non-Functional Requirements

- Reliability: The application should produce correct and consistent results.
- Maintainability: The source code should be organized and easy to update.
- Security: Sensitive information should be protected, and secure development practices should be followed.
- Efficiency: Automated testing should reduce manual effort and help identify errors early.

5. FLOW CHART



UGC AUTONOMOUS

6. ALGORITHM

The algorithm describes the sequence of activities involved in applying architecture rules of thumb to a DevOps workflow. It begins with code development and continues through version control, automated testing, and optional building and deployment. If testing fails, the developer corrects the errors and reruns the pipeline. If testing succeeds, the application proceeds to the next stage. Monitoring helps identify problems and maintain application reliability. This process reduces manual effort, improves software quality, and supports efficient software delivery.

Step 1: Start the process.

Step 2: The developer writes or updates the application source code using Python.

Step 3: Upload the source code to the GitHub repository using Git.

Step 4: GitHub Actions automatically triggers the CI pipeline whenever code is pushed or a pull request is created.

Step 5: Install the required dependencies and execute automated tests using Pytest.

Step 6: Check whether all tests pass. If any test fails, display the error, correct the code, and repeat the process from Step 2.

Step 7: If all tests pass, build the application and optionally package it using Docker.

Step 8: Deploy the application to the required environment, if deployment is configured.

Step 9: Monitor application health, errors, logs, and performance to ensure reliable operation.

Step 10: Apply architecture rules of thumb, including automation, modular design, security, early testing, consistent environments, and monitoring throughout the workflow.

Step 11: End the process.

UGC AUTONOMOUS

7. SOURCE CODE

App.py

```
def calculate_total(price, quantity):
    if price < 0 or quantity < 0:
        raise ValueError("Values cannot be negative")
    return price * quantity
def display_message():
    print("Architecture Rules of Thumb in DevOps")
    print("Automation improves software reliability.")
if __name__ == "__main__":
    display_message()
    total = calculate_total(100, 3)
    print("Total Amount:", total)
```

Automated Testing — test_app.py

```
import pytest

from app import calculate_total

def test_calculate_total():

    assert calculate_total(100, 3) == 300

def test_zero_quantity():

    assert calculate_total(100, 0) == 0

def test_negative_price():

    with pytest.raises(ValueError):

        calculate_total(-100, 2)

def test_negative_quantity():

    with pytest.raises(ValueError):
```

```
    calculate_total(100, -2)
```

CI/CD Pipeline — .github/workflows/ci.yml

```
name: DevOps CI Pipeline
on:
  push:
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install Dependencies
        run: pip install pytest
      - name: Run Automated Tests
        run: python -m pytest -v
```

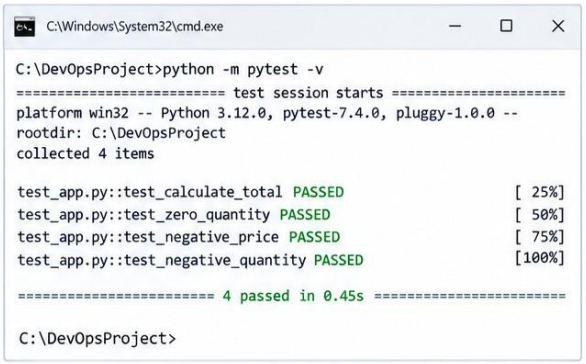
Execution Steps

1. Install Python and Git on your computer.
2. Create a project folder and save app.py and test_app.py inside it.
3. Install Pytest using commands.

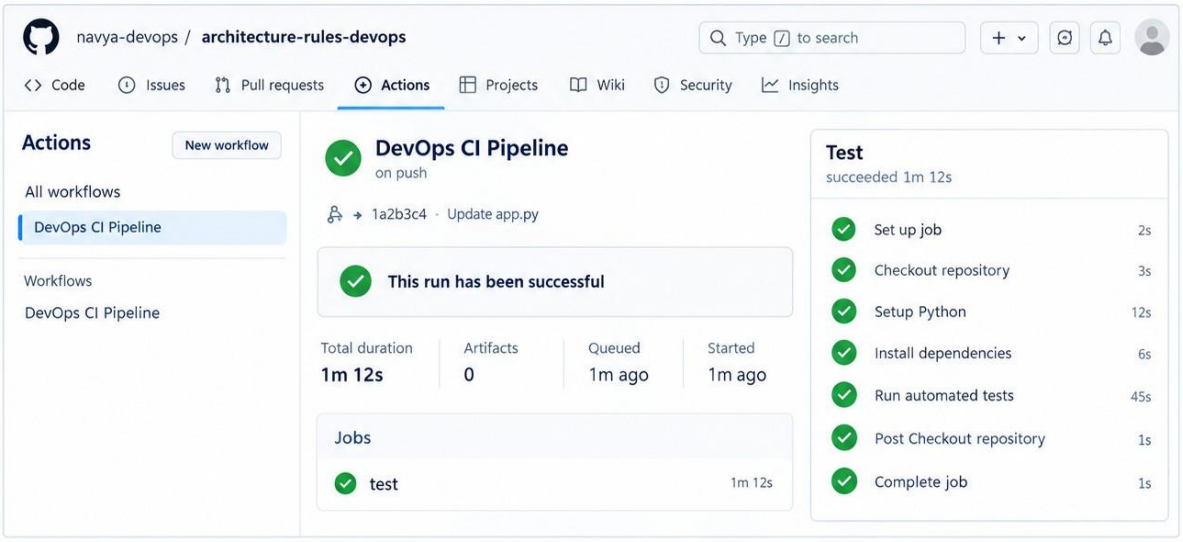
8. EXPERIMENTAL RESULTS



Output of app.py (Python Application)



Output of pytest (Automated Tests)



GitHub Actions Result (Successful Build and Tests)

UGC AUTONOMOUS

9. CONCLUSION

The project “Architecture Rules of Thumb in DevOps” demonstrates how architectural principles and DevOps practices can improve the software development and deployment process. By using Python, GitHub, GitHub Actions, and Pytest, the project shows how source code can be managed, tested automatically, and checked for errors whenever changes are made. The implementation highlights important principles such as automation, modular design, early testing, security, and reliability.

The automated testing process helps identify errors quickly and reduces manual effort. GitHub Actions provides a continuous integration workflow that improves the efficiency and consistency of software development. The project also explains how Docker, monitoring tools, and automated deployment can be integrated to enhance the system further.

In conclusion, applying architecture rules of thumb in DevOps helps create software systems that are reliable, maintainable, secure, and efficient. This project provides a basic understanding of CI/CD automation and establishes a foundation for implementing more advanced DevOps practices in real-world applications.

TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

10. FUTURE ENHANCEMENT

The project “Architecture Rules of Thumb in DevOps” can be further improved by incorporating advanced tools, automation techniques, and modern software architecture practices. These enhancements can make the system more secure, scalable, reliable, and suitable for real-world applications.

In the future, the existing CI pipeline can be extended into a complete CI/CD pipeline by adding automated deployment. Whenever developers upload updated code, the system can automatically build, test, and deploy the application to a staging or production environment. This reduces manual work and speeds up the software delivery process.

Containerization can be implemented using Docker to package the application and its dependencies into a consistent environment. Kubernetes can be introduced to manage containers, support automatic scaling, and improve application availability. These technologies help applications run consistently across development, testing, and production environments.

Security can be strengthened by integrating automated vulnerability scanning, dependency checking, secret detection, and code quality analysis into the pipeline. These practices help identify security risks before software is released. Role-based access control and secure secret management can also be implemented to protect sensitive information and restrict unauthorized access.

The project can additionally incorporate microservices architecture, automated rollback mechanisms, cloud integration, and disaster recovery strategies. Microservices can separate application functionality into smaller components, while automated rollback can restore a previous stable version when a deployment fails. Backup and recovery mechanisms can help protect application data and reduce downtime.

Finally, the project can be expanded to support multiple developers, multiple repositories, and larger software applications. By implementing these enhancements, Architecture Rules of Thumb in DevOps can evolve from a basic automated testing demonstration into a comprehensive software delivery system that follows modern architectural principles and DevOps best practices.

11. REFERENCES

1. GitHub, GitHub Actions Documentation. Available at: https://docs.github.com/en/actions.

GitHub Docs

2. GitHub, Building and Testing Python with GitHub Actions. Available at: https://docs.github.com/en/actions/tutorials/build-and-test-code/python.

GitHub Docs

3. Python Software Foundation, Python Documentation. Available at: https://docs.python.org/3/.

4. pytest Development Team, pytest Documentation. Available at: https://docs.pytest.org/.

5. Docker, Docker Documentation. Available at: https://docs.docker.com/.

6. HashiCorp, Terraform Documentation. Available at: https://developer.hashicorp.com/terraform/docs.

7. Prometheus, Prometheus Documentation. Available at: https://prometheus.io/docs/.

8. Grafana Labs, Grafana Documentation. Available at: https://grafana.com/docs/.

9. Amazon Web Services, DevOps Guidance and Best Practices. Available at: https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/devops-pattern-list.html.

AWS Prescriptive Guidance

10. Microsoft, Deploy with Azure Infrastructure as Code and GitHub Actions. Available at: https://learn.microsoft.com/devops/deliver/iac-github-actions.